

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

ЗВІТ

з лабораторної роботи №1
з дисципліни «Проектування програмних систем для мобільних пристроїв»

«Розробка WEB-Додатків для мобільних пристроїв»

Варіант №4

Виконав:
студент 2 курсу, групи ІМ-51мн
Ковальов Олександр

Перевірив:
асистент, Нестерук А. О.

Дата здачі: 24.09.2026

Мета. Ознайомитися з основними поняттями та особливостями розробки web-додатків для мобільних пристроїв.

Завдання. Розробити Android-додаток згідно з варіантом. Запустити програму на емуляторі. Виділити ключові особливості синтаксису мови.

Завдання за варіантом. Доменна область: Абітурієнт.

Поля: ID, Прізвище, Ім'я, По-батькові, Адреса, Телефон, Оцінки.

Створити масив об'єктів. Вивести:

- список абітурієнтів, які мають незадовільні оцінки;
- список абітурієнтів, середній бал у яких вище заданого;
- вибрати задане число N абітурієнтів, що мають найбільш високий середній бал (вивести також повний список абітурієнтів, які мають напівпрохідний бал).

Хід роботи.

Для розробки додатку була обрана мова програмування Rust. Так як застосунок повинен бути веб-орієнтованим, був обраний фреймворк Tauri – аналог Electron, але написаний повністю на Rust, через що швидкодія та використання ресурсів набагато кращі. Звичайно, перевага не лише в мові програмування – на відміну від Electron, Tauri не пакує двигун Chromium, а перевикористовує системний WebView.

Так як я використовую операційну систему Linux Fedora 44, налаштування оточення для програмування не буде занадто важким. В якості IDE будуть використовуватися RustRover та Android Studio. Після встановлення IDE треба встановити Rust – а саме утиліту `rustup`, менеджер тулчейнів. Вона завантажить і компілятор `rustc`, і менеджер пакетів `Cargo`.

Для роботи Tauri треба мати встановлені системні залежності. Їх можна встановити командою:

```
1 | sudo dnf install webkit2gtk4.1-devel openssl-devel curl wget file libappindicator-gtk3-devel  
   ↪ librsvg2-devel
```

Також, треба встановити рантайм двигун для Javascript. Є кілька варіантів – Node.js, Deno, Bun.

Node.js є класикою – це стандартний двигун, має найбільшу сумісність з Tauri. Розробники гарантують, що всі шаблони, плагіни Vite та пакети npm будуть працювати з коробки без проблем. Недоліками є повільніша інсталяція залежностей, великий каталог `node_modules`, тощо.

Вун, в свою чергу, є найновішим варіантом, найкращим для локальної розробки – мова йде саме про швидкість за бенчмарками. Також, має Hot-Reload для TypeScript. Але – повну сумісність з Tauri ніхто не обіцяє.

Щодо Deno, то в нього велика купа плюсів, таких як наявність пісочниці, внутрішнього тулінгу який дозволяє не засмічувати репозиторій, тощо. Але, знову ж, повної сумісності немає.

Оскільки я обираю інструменти в першу чергу розраховуючи на роботу з Rust та Tauri, обрав стандартний варіант – Node.js. Треба його встановити. Для цього використовується дві утиліти – `nvm` та `fnm`. Вибір пав на останній, оскільки він не вбудовується в `shell`, а поширюється як повноцінний бінарник. Тож, він був завантажений за інструкцією з [GitHub](#).

Встановлюємо LTS версію Node.js:

```
1 | fnm install 24
```

Наступним кроком є вибір менеджера пакетів для Node.js. Їх на вибір теж декілька, у кожного свої переваги та недоліки. Серед npm, yarn, та pnpm вибір знову пав на останній – оскільки пакети перевикористовуються глобально, тож немає відомої проблеми з вагою node_modules (хоч і локально, але не архітектурно), та й загалом за бенчмарками він працює швидше. Тож, встановлюємо:

[illegible]

Далі, встановлюємо Tauri.

```
1 | cargo install tauri-cli --version "\^2" --locked
```

Створюємо каталог Lab1, а в ній проект, за допомогою команди, вказаної на скріншоті. В якості мови фронтенду був обраний TypeScript, а бібліотеки для UI – React.

```
xairaven@laptop ~/Files/Programming/KPI-DSSMD <main>
$ pnpm create tauri-app@latest .
✓ Package name · lab1
✓ Identifier · com.xairaven.lab1
✓ Choose which language to use for your frontend · TypeScript / JavaScript
✓ Choose your package manager · pnpm
✓ Choose your UI template · React - (https://react.dev/)
✓ Choose your UI flavor · TypeScript

Template created! To get started run:
  pnpm install
  pnpm tauri android init

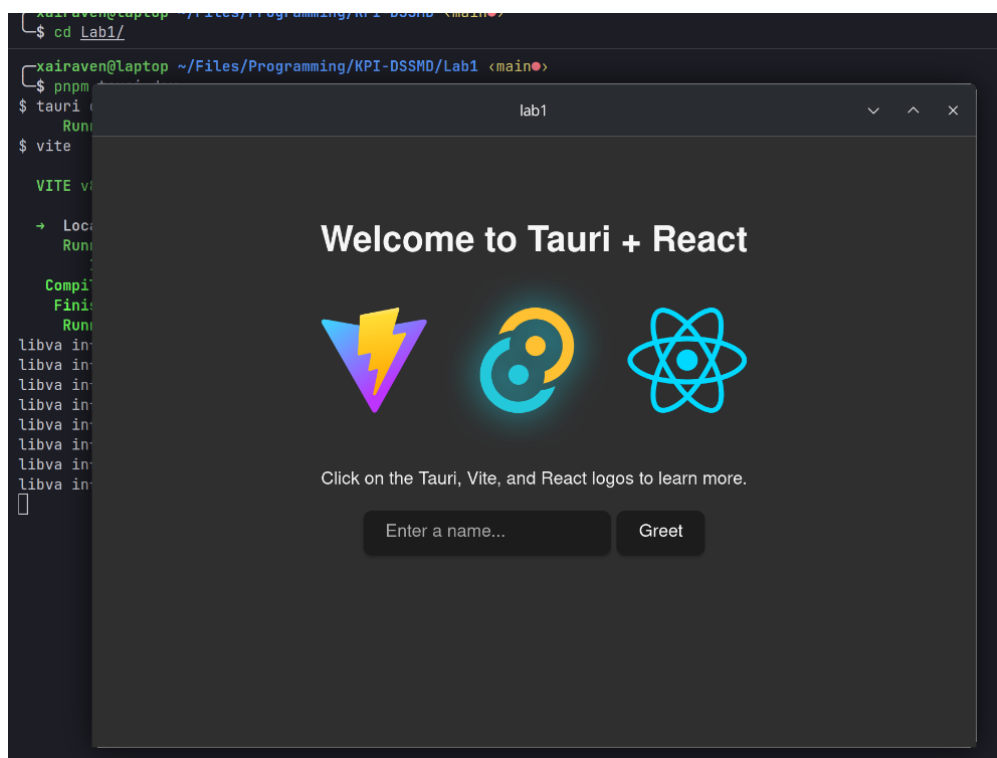
For Desktop development, run:
  pnpm tauri dev

For Android development, run:
  pnpm tauri android dev
```

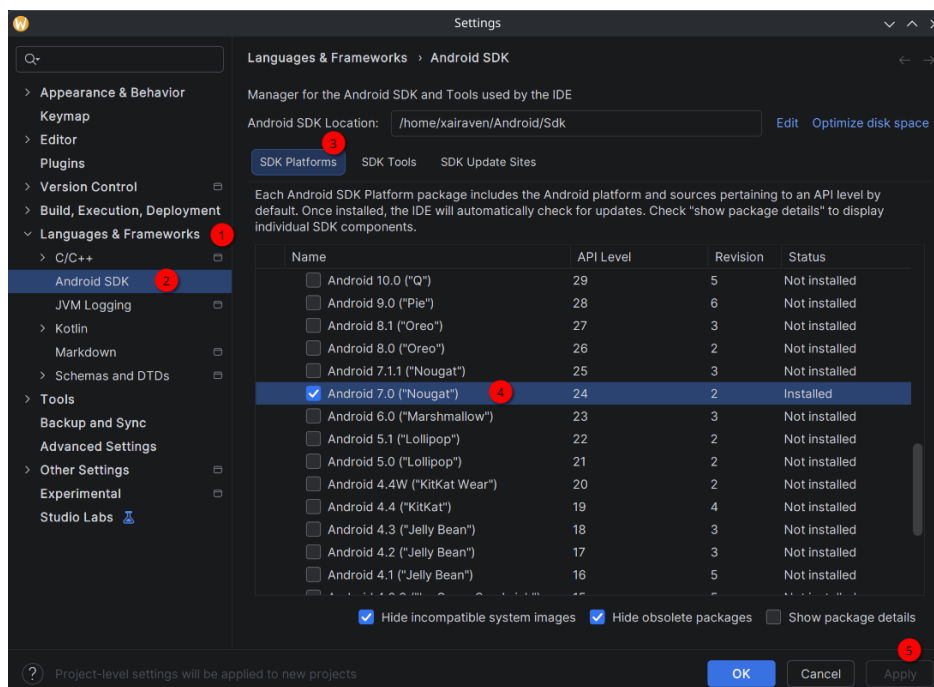
Виконуємо дві команди – встановлення всіх залежностей, та тестовий запуск десктопного застосунку в режимі розробника.

```
1 | pnpm install
2 | pnpm tauri dev
```

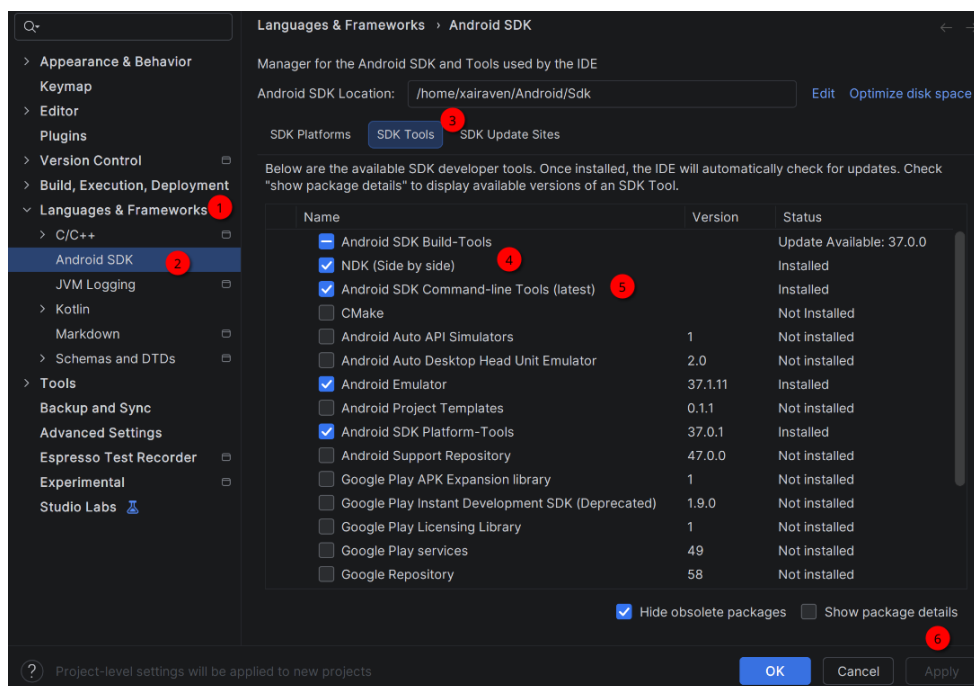
Бачимо вікно додатку, створеного з шаблону. Перший етап перевірки пройшов успішно.



Другий етап налаштування передбачає підготовку середовища для розробки Android застосунку. Встановлюємо Android Studio. Першим кроком є завантаження Android SDK. Обираємо базовий варіант – Android 7.0 «Nougat», API Level 24, який є найпоширенішим.



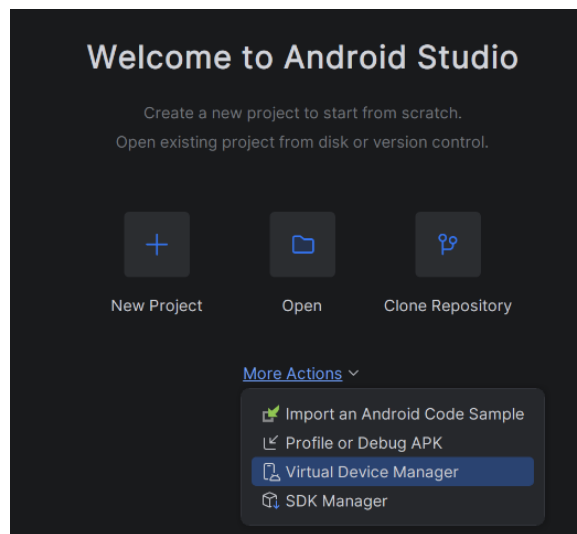
Далі, завантажуюємо NDK (Native-development Kit) і SDK CLI Tools. Перший потрібен щоб писати частини застосунку на нативних мовах програмування як С чи С++ – тобто саме те, що буде робити Таугі під капотом. Друге потрібно для роботи з компонентами набору розробки через консоль, без використання Android Studio.



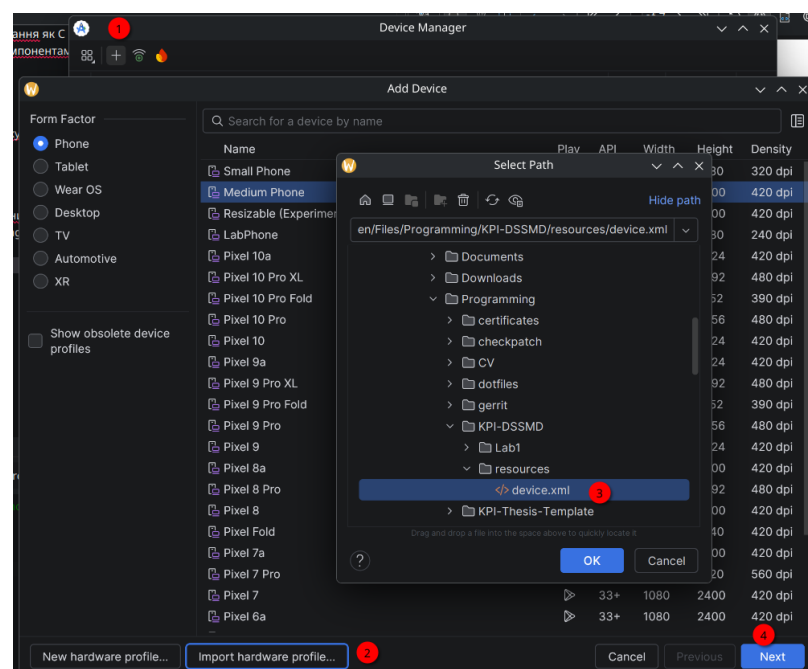
Встановлюємо змінні оточення в оболонку, щоб можна було звертатися до утиліт не за абсолютним шляхом, а лише за назвою.

```
# Android
export ANDROID_HOME="$HOME/Android/Sdk"
export NDK_HOME="$ANDROID_HOME/ndk/30.0.16248370"
export PATH="$PATH:$ANDROID_HOME/platform-tools:$ANDROID_HOME/emulator"
```

Далі треба створити емулятор – віртуальний пристрій. На привітальній сторінці Android Studio обираємо «More Actions» та «Virtual Device Manager»:



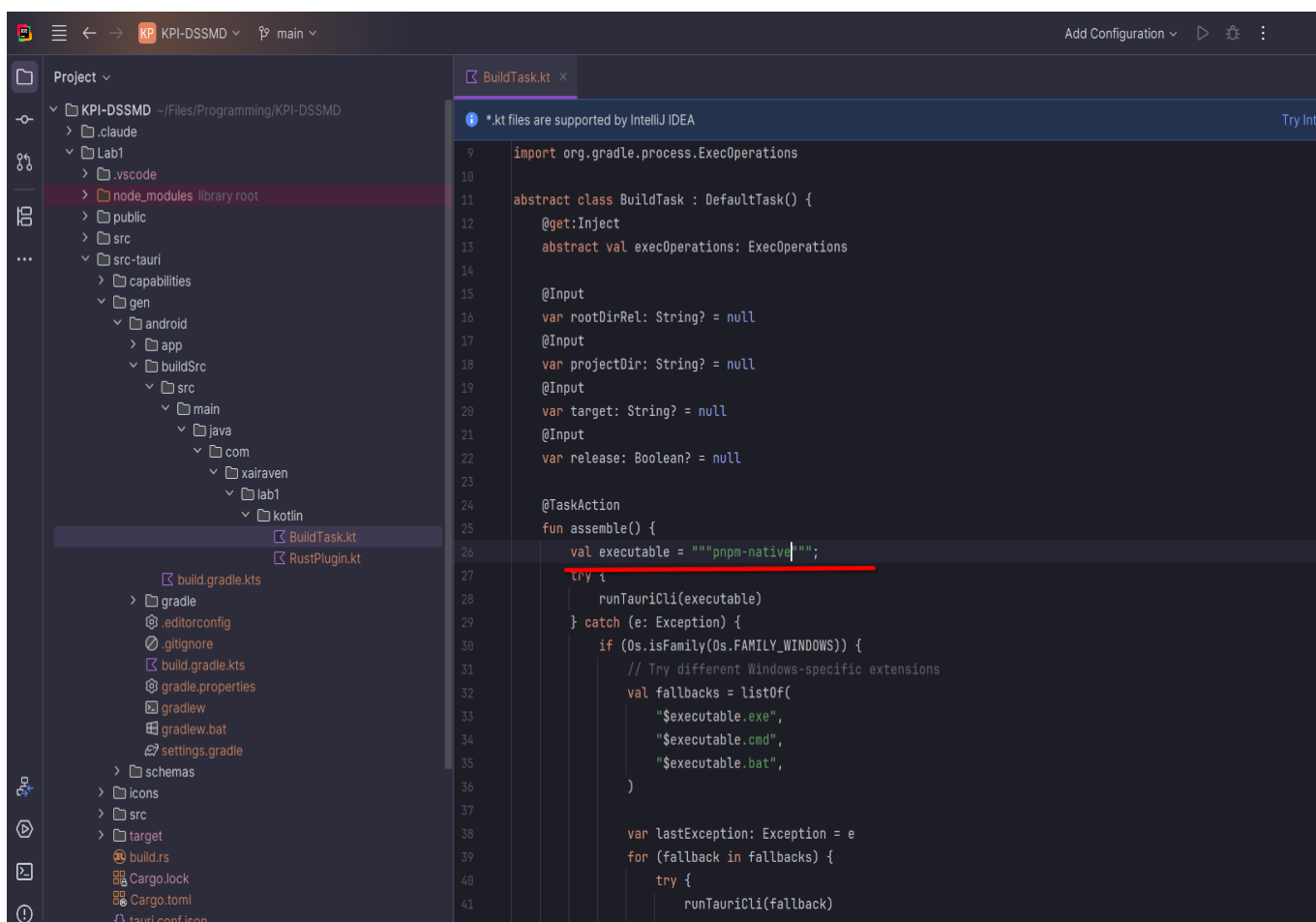
У вікні натискаємо на плюсики, і потім імпортуємо заздалегідь збережений файл з описом пристрою, який залишився з минулої вибіркової дисципліни – «Сучасні мобільні операційні системи», і наразі знаходиться в каталозі resources проекту.



Далі, в проєкті запускаємо команду, яка додасть відповідні налаштування для розробки під Android. Після цього запускаємо тестовий проєкт на емуляторі.

```
1 | npm tauri android init
2 | npm tauri android dev
```

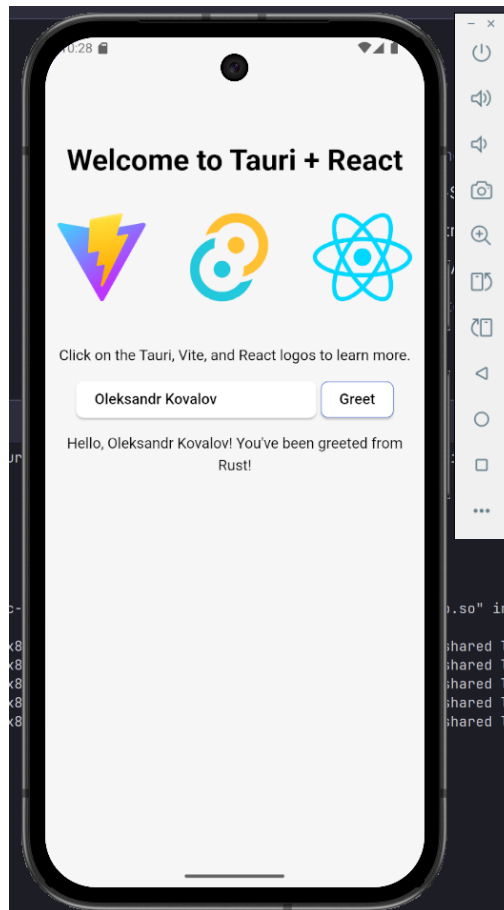
В процесі виникає декілька помилок. По-перше, на системі встановлена JDK 25, що є занадто високою версією для деяких зі встановлених раніше утиліт. Проблема вирішилась встановленням JDK 21. Також, виникла помилка при генерації Kotlin скриптів – Tauri вказує викликаючу утиліту як `npm-native`, хоча викликати-ся повинен саме `npm`. Через це остання вказана команда не відпрацьовує. Наразі, щоб виправити цю помилку, в скрипті треба вказати просто `npm`. Це треба робити кожного разу, коли відбувається ініціалізація Android проєкту, що відбувається нечасто. Я розробив виправлення проблеми та надіслав [Pull Request](#) розробникам. Зміни були прийняті, тож скоро такої проблеми виникати не буде.



Також, виникли ще деякі помилки. Спочатку процес `adb` (Android Debug Bridge) падав через відому помилку в `mDNS`. Це можна виправити, якщо встановити `environmental variable`:

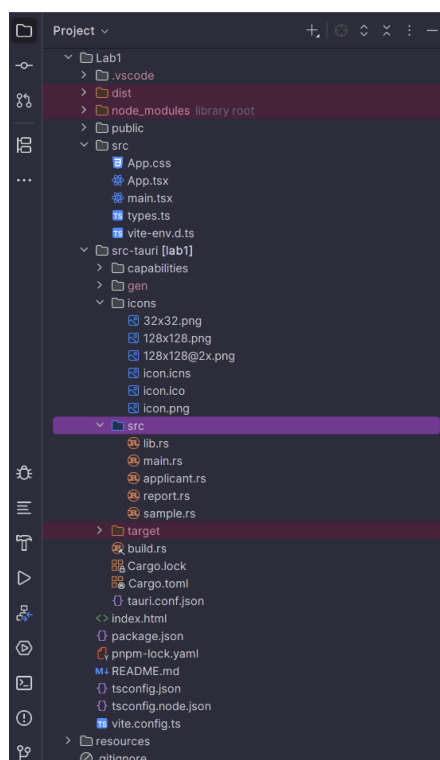
```
1 | export ADB_MDNS=0
```

Шаблон застосунку для Android готовий.



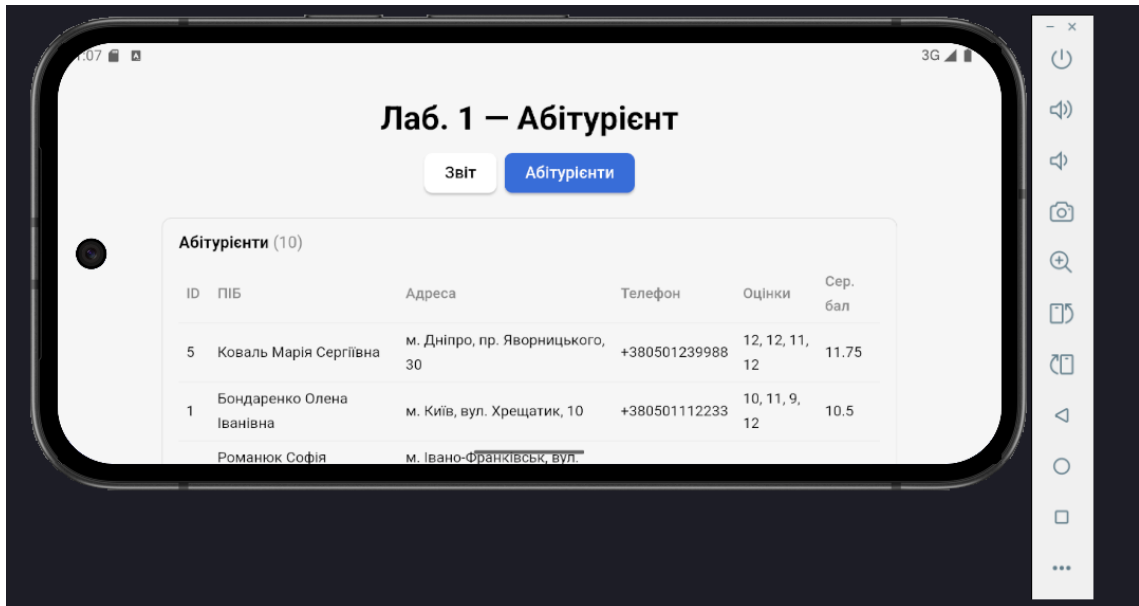
Нарешті, переходимо безпосередньо до розробки застосунку за доменною областю.

На скріншоті продемонстрована файлова структура готового додатку:

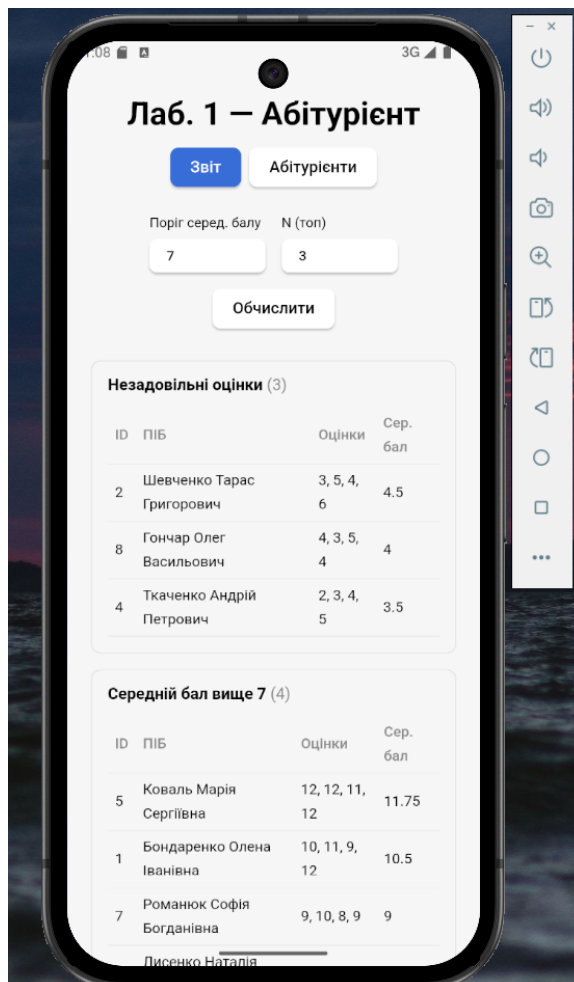


Код на Rust виступає основним обробником даних. Були створені відповідні структури та методи до них. Код написаний на TypeScript та CSS відповідає за оформлення та функціональність інтерфейсу. Додаток має дві сторінки: загальна інформація та звіт.

Сторінка з інформацією:



Сторінка зі звітом:



Аналізуючи програмний код розробленого застосунку, можна виділити кілька ключових синтаксичних особливостей використаних мов. У частині бекенду, написаній на Rust, яскраво виражена строга статична типізація та використання системи власності з позичанням за допомогою посилань, що гарантує безпеку пам'яті. Широко застосовуються макроси, зокрема для автоматичної генерації реалізацій трейтів серіалізації та клонування, а також функціональний підхід з використанням замикань та ітераторів для фільтрації масивів даних. Фронтенд частина, написана на TypeScript, демонструє розширення класичного JavaScript строгою типізацією через використання інтерфейсів. Також застосовується синтаксис TSX, який дозволяє декларативно описувати структуру користувацького інтерфейсу безпосередньо в коді разом з логікою управління станом компонентів.

Висновок. Під час виконання лабораторної роботи було успішно розроблено мобільний додаток для ОС Android з використанням фреймворку Tauri. Було налаштовано середовище розробки, зокрема Node.js, pnpm, Rust, Android SDK та NDK. Програмна частина реалізована за допомогою Rust для обробки даних та TypeScript з React для побудови користувацького інтерфейсу. Додаток успішно компілюється та запускається на емуляторі Android, реалізуючи весь необхідний функціонал для обробки даних абітурієнтів, включаючи фільтрацію за оцінками та сортування.

Контрольні запитання.

1. Дайте визначення поняття web-додатку.

Web-додаток є програмним застосунком, де клієнтом виступає браузер або вбудований WebView, а логіка та інтерфейс реалізовані веб-технологіями, такими як HTML, CSS та JavaScript.

2. Назвіть основні відмінності між мобільними і desktop додатками.

Головні відмінності між мобільними та десктопними додатками полягають в адаптації інтерфейсу під невеликі сенсорні екрани, жорсткіший оптимізації споживання енергії та ресурсів пам'яті, а також у тіснішій взаємодії з апаратними датчиками пристрою.

3. Наведіть види мобільних web-додатків та охарактеризуйте їх.

Мобільні веб-додатки здебільшого поділяються на прогресивні застосунки (PWA), які виконуються в браузері з можливістю офлайн-роботи, та гібридні додатки, де веб-технології запаковані в нативний контейнер із системним доступом до API смартфона.

4. Перелічіть сучасні об'єктно-орієнтовані мови програмування.

Серед популярних сучасних об'єктно-орієнтованих мов програмування можна виділити Java, C#, C++, Python, TypeScript, Swift, Ruby та Kotlin.

Лістинг.

main.rs

```
1 // Prevents additional console window on Windows in release, DO NOT REMOVE!!
2 #![cfg_attr(not(debug_assertions), windows_subsystem = "windows")]
3
4 fn main() {
5     lab1_lib::run();
6 }
```

lib.rs

```
1 #[cfg_attr(mobile, tauri::mobile_entry_point)]
2 pub fn run() {
3     if let Err(err) = tauri::Builder::default()
4         .plugin(tauri_plugin_opener::init())
5         .invoke_handler(tauri::generate_handler![crate::report::build_report])
6         .run(tauri::generate_context!())
7     {
8         eprintln!("Error occurred while running Tauri application: {err}");
9         std::process::exit(1);
10    }
11 }
12
13 mod applicant;
14 mod report;
15 mod sample;
```

applicant.rs

```
1 use serde::Serialize;
2
3 #[derive(Debug, Clone, Serialize)]
4 pub struct Applicant {
5     pub id: u32,
6     pub last_name: String,
7     pub first_name: String,
8     pub patronymic: String,
9     pub address: String,
10    pub phone: String,
11    pub grades: Vec<u8>,
12 }
13
14 pub const PASSING_GRADE: u8 = 4;
15 pub const BORDERLINE_RANGE: std::ops::Range<f64> = 4.0..6.0;
16
17 impl Applicant {
18     pub fn average_grade(&self) -> f64 {
19         if self.grades.is_empty() {
20             return 0.0;
21         }
22         let sum = f64::from(self.grades.iter().sum::<u8>());
23         #[allow(clippy::cast_precision_loss)]
24         let avg = sum / self.grades.len() as f64;
25
26         (avg * 100.0).round() / 100.0
27     }
28
29     pub fn has_failing_grade(&self) -> bool {
30         self.grades.iter().any(|&g| g < PASSING_GRADE)
```

```

31     }
32
33     pub fn is_above_average(&self, threshold: f64) -> bool {
34         self.average_grade() > threshold
35     }
36
37     pub fn is_borderline(&self) -> bool {
38         BORDERLINE_RANGE.contains(&self.average_grade())
39     }
40 }
41
42 #[derive(Debug, Clone, Serialize)]
43 pub struct ApplicantView {
44     #[serde(flatten)]
45     pub applicant: Applicant,
46     pub average: f64,
47 }
48
49 impl From<&Applicant> for ApplicantView {
50     fn from(a: &Applicant) -> Self {
51         Self {
52             applicant: a.clone(),
53             average: a.average_grade(),
54         }
55     }
56 }
57
58 #[cfg(test)]
59 mod tests {
60     use super::Applicant;
61     use super::BORDERLINE_RANGE;
62     use crate::sample;
63
64     fn applicant(grades: Vec<u8>) -> Applicant {
65         Applicant {
66             id: 0,
67             last_name: "Kovalov".into(),
68             first_name: "Alex".into(),
69             patronymic: "Oleksiyovuch".into(),
70             address: "Somewhere".into(),
71             phone: "Idk?".into(),
72             grades,
73         }
74     }
75
76     #[test]
77     fn average_grade_rounds_to_two_decimals() {
78         assert_eq!(applicant(vec![7, 8, 9]).average_grade(), 8.0);
79         assert_eq!(applicant(vec![5, 6]).average_grade(), 5.5);
80     }
81
82     #[test]
83     fn detects_failing_grades() {
84         assert!(applicant(vec![3, 10, 10]).has_failing_grade());
85         assert!(!applicant(vec![4, 10, 10]).has_failing_grade());
86     }
87
88     #[test]
89     fn filters_above_threshold() {
90         let applicants = sample::get_applicants();
91         let threshold = 9.0;
92
93         let result: Vec<Applicant> = applicants
94             .into_iter()

```

```

95         .filter(|a| a.is_above_average(threshold))
96         .collect();
97
98         assert!(result.iter().all(|a| a.average_grade() > 9.0));
99     }
100
101     #[test]
102     fn top_n_is_sorted_descending_and_limited() {
103         let mut applicants = sample::get_applicants();
104         let top_n = 3;
105
106         applicants.sort_by(|a, b| {
107             let a = a.average_grade();
108             let b = b.average_grade();
109             b.total_cmp(&a)
110         });
111         let result: Vec<Applicant> = applicants.into_iter().take(top_n).collect();
112
113         assert_eq!(result.len(), top_n);
114         assert!(result[0].average_grade() >= result[1].average_grade());
115         assert!(result[1].average_grade() >= result[2].average_grade());
116     }
117
118     #[test]
119     fn borderline_stays_within_range() {
120         let applicants = sample::get_applicants();
121
122         let result: Vec<Applicant> = applicants
123             .into_iter()
124             .filter(|a| a.is_borderline())
125             .collect();
126
127         assert!(
128             result
129                 .iter()
130                 .all(|a| BORDERLINE_RANGE.contains(&a.average_grade()))
131         );
132     }
133 }

```

report.rs

```

1  use crate::applicant::ApplicantView;
2  use crate::sample;
3  use serde::Serialize;
4
5  #[derive(Debug, Clone, Serialize)]
6  pub struct Report {
7      pub all: Vec<ApplicantView>,
8      pub failing: Vec<ApplicantView>,
9      pub above_threshold: Vec<ApplicantView>,
10     pub top_n: Vec<ApplicantView>,
11     pub borderline: Vec<ApplicantView>,
12 }
13
14 #[tauri::command]
15 pub fn build_report(threshold: f64, top_count: usize) -> Report {
16     let mut applicants = sample::get_applicants();
17
18     applicants.sort_by(|a, b| {
19         let a = a.average_grade();
20         let b = b.average_grade();
21         b.total_cmp(&a)

```

```

22     });
23
24     Report {
25         all: applicants.iter().map(ApplicantView::from).collect(),
26         failing: applicants
27             .iter()
28             .filter(|a| a.has_failing_grade())
29             .map(ApplicantView::from)
30             .collect(),
31         above_threshold: applicants
32             .iter()
33             .filter(|a| a.is_above_average(threshold))
34             .map(ApplicantView::from)
35             .collect(),
36         top_n: applicants
37             .iter()
38             .take(top_count)
39             .map(ApplicantView::from)
40             .collect(),
41         borderline: applicants
42             .iter()
43             .filter(|a| a.is_borderline())
44             .map(ApplicantView::from)
45             .collect(),
46     }
47 }

```

sample.rs

```

1 use crate::applicant::Applicant;
2
3 pub fn get_applicants() -> Vec<Applicant> {
4     vec![
5         Applicant {
6             id: 1,
7             last_name: "Бондаренко".into(),
8             first_name: "Олена".into(),
9             patronymic: "Іванівна".into(),
10            address: "м. Київ, вул. Хрещатик, 10".into(),
11            phone: "+380501112233".into(),
12            grades: vec![10, 11, 9, 12],
13        },
14        Applicant {
15            id: 2,
16            last_name: "Шевченко".into(),
17            first_name: "Тарас".into(),
18            patronymic: "Григорович".into(),
19            address: "м. Львів, вул. Франка, 5".into(),
20            phone: "+380671234567".into(),
21            grades: vec![3, 5, 4, 6],
22        },
23        Applicant {
24            id: 3,
25            last_name: "Мельник".into(),
26            first_name: "Ірина".into(),
27            patronymic: "Олексіївна".into(),
28            address: "м. Одеса, вул. Дерибасівська, 22".into(),
29            phone: "+380931112244".into(),
30            grades: vec![6, 7, 5, 8],
31        },
32        Applicant {
33            id: 4,
34            last_name: "Ткаченко".into(),

```

```

35     first_name: "Андрій".into(),
36     patronymic: "Петрович".into(),
37     address: "м. Харків, вул. Сумська, 15".into(),
38     phone: "+380661234455".into(),
39     grades: vec![2, 3, 4, 5],
40 },
41 Applicant {
42     id: 5,
43     last_name: "Коваль".into(),
44     first_name: "Марія".into(),
45     patronymic: "Сергіївна".into(),
46     address: "м. Дніпро, пр. Яворницького, 30".into(),
47     phone: "+380501239988".into(),
48     grades: vec![12, 12, 11, 12],
49 },
50 Applicant {
51     id: 6,
52     last_name: "Кравець".into(),
53     first_name: "Віктор".into(),
54     patronymic: "Миколайович".into(),
55     address: "м. Запоріжжя, вул. Соборна, 8".into(),
56     phone: "+380671239900".into(),
57     grades: vec![7, 6, 7, 6],
58 },
59 Applicant {
60     id: 7,
61     last_name: "Романюк".into(),
62     first_name: "Софія".into(),
63     patronymic: "Богданівна".into(),
64     address: "м. Івано-Франківськ, вул. Незалежності, 3".into(),
65     phone: "+380931238877".into(),
66     grades: vec![9, 10, 8, 9],
67 },
68 Applicant {
69     id: 8,
70     last_name: "Гончар".into(),
71     first_name: "Олег".into(),
72     patronymic: "Васильович".into(),
73     address: "м. Полтава, вул. Соборності, 12".into(),
74     phone: "+380661237766".into(),
75     grades: vec![4, 3, 5, 4],
76 },
77 Applicant {
78     id: 9,
79     last_name: "Лисенко".into(),
80     first_name: "Наталія".into(),
81     patronymic: "Ігорівна".into(),
82     address: "м. Чернігів, вул. Шевченка, 7".into(),
83     phone: "+380501236655".into(),
84     grades: vec![8, 7, 9, 7],
85 },
86 Applicant {
87     id: 10,
88     last_name: "Захарчук".into(),
89     first_name: "Дмитро".into(),
90     patronymic: "Олександрович".into(),
91     address: "м. Вінниця, вул. Соборна, 20".into(),
92     phone: "+380671235544".into(),
93     grades: vec![5, 6, 5, 6],
94 },
95 ]
96 }

```

```

1 import React from "react";
2 import ReactDOM from "react-dom/client";
3 import App from "./App";
4
5 ReactDOM.createRoot(document.getElementById("root") as HTMLElement).render(
6   <React.StrictMode>
7     <App />
8   </React.StrictMode>,
9   );
10

```

App.tsx

```

1 import { useEffect, useState } from "react";
2 import { invoke } from "@tauri-apps/api/core";
3 import type { Applicant, Report } from "./types";
4 import "./App.css";
5
6 function ApplicantTable({ title, rows }: { title: string; rows: Applicant[] }) {
7   return (
8     <section className="panel">
9       <h2>
10        {title} <span className="count">({rows.length})</span>
11      </h2>
12      {rows.length === 0 ? (
13        <p className="empty">❏ немає ❏</p>
14      ) : (
15        <table>
16          <thead>
17            <tr>
18              <th>ID</th>
19              <th>ПІБ</th>
20              <th>Оцінки</th>
21              <th>Сер. бал</th>
22            </tr>
23          </thead>
24          <tbody>
25            {rows.map((a) => (
26              <tr key={a.id}>
27                <td>{a.id}</td>
28                <td>
29                  {a.last_name} {a.first_name} {a.patronymic}
30                </td>
31                <td>{a.grades.join(", ")}</td>
32                <td>{a.average}</td>
33              </tr>
34            ))}
35          </tbody>
36        </table>
37      )}
38    </section>
39  );
40 }
41
42 function StudentsPage({ rows }: { rows: Applicant[] }) {
43   return (
44     <section className="panel">
45       <h2>
46        Абітурієнти <span className="count">({rows.length})</span>
47      </h2>
48      <table>
49        <thead>
50          <tr>

```

```

51     <th>ID</th>
52     <th>ПІБ</th>
53     <th>Адреса</th>
54     <th>Телефон</th>
55     <th>Оцінки</th>
56     <th>Сер. бал</th>
57 </tr>
58 </thead>
59 <tbody>
60 {rows.map((a) => (
61     <tr key={a.id}>
62     <td>{a.id}</td>
63     <td>
64     {a.last_name} {a.first_name} {a.patronymic}
65     </td>
66     <td>{a.address}</td>
67     <td>{a.phone}</td>
68     <td>{a.grades.join(", ")}</td>
69     <td>{a.average}</td>
70     </tr>
71     ))}
72 </tbody>
73 </table>
74 </section>
75 );
76 }
77
78 function ReportPage({ report }: { report: Report | null }) {
79     // Kept as plain strings (not numbers) so the input always shows exactly
80     // what was typed.
81     const [thresholdInput, setThresholdInput] = useState("7");
82     const [topCountInput, setTopCountInput] = useState("3");
83     const [computed, setComputed] = useState<Report | null>(null);
84
85     async function compute() {
86         const threshold = Number(thresholdInput) || 0;
87         const topCount = Number(topCountInput) || 0;
88         setComputed(await invoke<Report>("build_report", { threshold, topCount }));
89     }
90
91     useEffect(() => {
92         if (report) setComputed(report);
93     }, [report]);
94
95     return (
96     <>
97     <form
98     className="controls"
99     onSubmit={(e) => {
100         e.preventDefault();
101         compute();
102     }}
103     >
104     <label>
105     Попір серед. балу
106     <input
107     type="number"
108     step="0.1"
109     value={thresholdInput}
110     onChange={(e) => setThresholdInput(e.currentTarget.value)}
111     />
112     </label>
113     <label>
114     N (тон)

```

```

115     <input
116     type="number"
117     min={1}
118     value={topCountInput}
119     onChange={(e) => setTopCountInput(e.currentTarget.value)}
120     />
121   </label>
122   <button type="submit">Обчислити</button>
123 </form>
124
125   {computed && (
126     <>
127     <ApplicantTable title="Незадовільні оцінки" rows={computed.failing} />
128     <ApplicantTable
129     title={`Середній бал вище ${thresholdInput}`}
130     rows={computed.above_threshold}
131     />
132     <ApplicantTable
133     title={`Топ-${topCountInput} за середнім балом`}
134     rows={computed.top_n}
135     />
136     <ApplicantTable title="Напівпрохідний бал" rows={computed.borderline} />
137     </>
138   )}
139 </>
140 );
141 }
142
143 function App() {
144   const [page, setPage] = useState<"report" | "students">("report");
145   const [report, setReport] = useState<Report | null>(null);
146
147   useEffect(() => {
148     invoke<Report>("build_report", { threshold: 7, topCount: 3 }).then(setReport);
149   }, []);
150
151   return (
152     <main className="container">
153     <h1>Лаб. 1  Абітурієнт</h1>
154
155     <nav className="nav">
156       <button
157       className={page === "report" ? "active" : ""}
158       onClick={() => setPage("report")}
159       >
160       Звіт
161       </button>
162       <button
163       className={page === "students" ? "active" : ""}
164       onClick={() => setPage("students")}
165       >
166       Абітурієнти
167       </button>
168     </nav>
169
170     {page === "report" ? (
171       <ReportPage report={report} />
172     ) : (
173       <StudentsPage rows={report?.all ?? []} />
174     )}
175     </main>
176   );
177 }
178

```

App.css

```

1  :root {
2      font-family: Inter, Avenir, Helvetica, Arial, sans-serif;
3      font-size: 16px;
4      line-height: 24px;
5      font-weight: 400;
6
7      color: #0f0f0f;
8      background-color: #f6f6f6;
9
10     font-synthesis: none;
11     text-rendering: optimizeLegibility;
12     -webkit-font-smoothing: antialiased;
13     -moz-osx-font-smoothing: grayscale;
14     -webkit-text-size-adjust: 100%;
15 }
16
17 .container {
18     margin: 0 auto;
19     max-width: 720px;
20     padding: 2rem 1rem 4rem;
21 }
22
23 h1 {
24     text-align: center;
25 }
26
27 .nav {
28     display: flex;
29     justify-content: center;
30     gap: 0.5rem;
31     margin-bottom: 1.5rem;
32 }
33
34 .nav button.active {
35     border-color: #396cd8;
36     background-color: #396cd8;
37     color: #ffffff;
38 }
39
40 .controls {
41     display: flex;
42     flex-wrap: wrap;
43     align-items: end;
44     gap: 1rem;
45     justify-content: center;
46     margin-bottom: 2rem;
47 }
48
49 .controls label {
50     display: flex;
51     flex-direction: column;
52     gap: 0.25rem;
53     font-size: 0.85em;
54 }
55
56 input,
57 button {
58     border-radius: 8px;
59     border: 1px solid transparent;

```

```

60     padding: 0.6em 1.2em;
61     font-size: 1em;
62     font-weight: 500;
63     font-family: inherit;
64     color: #0f0f0f;
65     background-color: #ffffff;
66     transition: border-color 0.25s;
67     box-shadow: 0 2px 2px rgba(0, 0, 0, 0.2);
68     outline: none;
69 }
70
71 input {
72     width: 6em;
73 }
74
75 button {
76     cursor: pointer;
77 }
78
79 button:hover {
80     border-color: #396cd8;
81 }
82 button:active {
83     border-color: #396cd8;
84     background-color: #e8e8e8;
85 }
86
87 .panel {
88     border: 1px solid #dcdcdc;
89     border-radius: 10px;
90     padding: 0.75rem 1rem 1rem;
91     margin-bottom: 1rem;
92     overflow-x: auto;
93 }
94
95 .panel h2 {
96     font-size: 1em;
97     margin: 0 0 0.6rem;
98 }
99
100 .count {
101     color: #8a8a8a;
102     font-weight: 400;
103 }
104
105 .empty {
106     color: #8a8a8a;
107     margin: 0;
108 }
109
110 table {
111     width: 100%;
112     border-collapse: collapse;
113     font-size: 0.9em;
114 }
115
116 th,
117 td {
118     text-align: left;
119     padding: 0.3em 0.5em;
120     border-bottom: 1px solid #eaeaea;
121 }
122
123 th {

```

```

124     color: #8a8a8a;
125     font-weight: 500;
126 }
127
128 @media (prefers-color-scheme: dark) {
129     :root {
130         color: #f6f6f6;
131         background-color: #2f2f2f;
132     }
133
134     input,
135     button {
136         color: #ffffff;
137         background-color: #0f0f0f98;
138     }
139     button:active {
140         background-color: #0f0f0f69;
141     }
142
143     .panel {
144         border-color: #4a4a4a;
145     }
146
147     th,
148     td {
149         border-bottom-color: #3f3f3f;
150     }
151 }

```

types.ts

```

1  export interface Applicant {
2      id: number;
3      last_name: string;
4      first_name: string;
5      patronymic: string;
6      address: string;
7      phone: string;
8      grades: number[];
9      average: number;
10 }
11
12 export interface Report {
13     all: Applicant[];
14     failing: Applicant[];
15     above_threshold: Applicant[];
16     top_n: Applicant[];
17     borderline: Applicant[];
18 }

```